

```
for(var i=0;i<channels;i++){
    var red_component_pixel = image.data[i*4 + 0];
    var green_component_pixel = image.data[i*4 + 1];
    var blue_component_pixel = image.data[i*4 + 2];
}
```

Computation of gray scale images

A gray scale image is one in which all colour components are normalised to have equal weightage. If an 8-bit image is considered, the colour gray is obtained when the number of RGB bits equals 1.

To solve this, there is a simple formula, which creates a weighted sum of pixel values to yield a gray image:

```
gray[pixel] = 0.21*red_component_pixel + 0.72*green_
component_pixel + 0.07*blue_component_pixel'
```

On applying the above formula to each pixel, split into its components, one gets an equivalent gray pixel.

Computation of binary and inverted images

A binary image is in black and white (BW). The conversion of an image from colour to BW is done through a process called thresholding, which classifies each pixel as white or black based on its value. If the value is greater than a particular threshold, it will be set to 255, else 0.

```
if(red_component_pixel > threshold_red &&
    green_component_pixel > threshold_green &&
    blue_component_pixel > threshold_blue){
    //make pixel == white
    image.data[pixel] = 255;
}else{ image.data[pixel] = 0; }
```

Just as we have negatives for a photograph, similarly, the inversion of colour space of any image converts all pixels into a negative. This can simply be done by subtracting each pixel value from 255.

The *tracking.js* library

According to GitHub, *tracking.js* is a lightweight JS library that offers a variety of computer vision algorithms with HTML5 and JS. Some algorithms implemented here are for colour tracking, face detection, feature descriptors and the other utility functions. To set up *tracking.js* for your Web page, include *build/tracking.js* inside your `<head>`. For more details, one can visit *tracking.js* documentation. It is highly detailed and illustrated.

Colour tracker using *tracking.js*: To initialise a colour tracker, first use the following commands:

```
var myTracker = new tracking.ColorTracker(['yellow']);
myTracker.on("track", color_tracking_callback);
var m1 = tracking.track("#myVideo", myTracker);
```

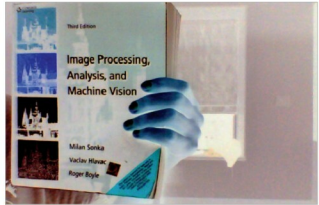


Figure 4: Inversion

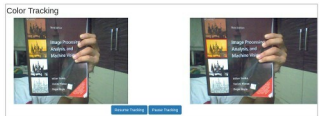


Figure 5: Yellow colour based tracking of the book



Figure 6: Multiple colour region tracking

In the above code snippet, *color_tracking_callback* is a callback which will receive a list of all possible locations where the given colour is present. Each location is a rectangle object, comprising attributes which are 'x', 'y', 'width' and 'height'. 'x' and 'y' are the starting points of the rectangle.

The natural action for tracking is to make a bounding box around the region we are interested in. Therefore, the *boundingBox* function plots a rectangle around the region of interest. Context variable is used here to perform any canvas drawing methods. *context.stroke()* eventually prints it on the canvas.

```
function color_tracking_callback(list_rect){
    list_rect.data.forEach(drawBoundingBox);
}
```

```
function drawBoundingBox(rect){
    context.beginPath();
    context.strokeStyle = "red";
    context.lineWidth = "2";
    context.rect(rect.x, rect.y, rect.width, rect.height);
```